TECHNICAL DOCUMENTATION

Arduino Measuring Tape

ESP32 + BLE distance meter — Expo / React Native Android app

Version: 1.0.0 · **Milestone:** Final application
Date: 25 October 2026 · **Repository:** Arduino Measuring Tape

This edition documents the final ESP32/BLE application, the Android transport, persistence, alerting and quality gates. The original Milestone 1 UNO/SPP material is retained later as historical context.

Contents

1. Final application (v1.0)
2. Historical project overview
3. Historical getting started
4. System architecture
5. Mobile application
6. Design system
7. Screens in detail
8. Firmware — Arduino UNO (historical)
9. Wire protocol — SPP (historical)
10. Quality & verification (historical)
11. Progress report — Milestone 1 (historical)
12. Roadmap
13. Appendix — tokens, glossary, file index

FINAL APPLICATION

The target is now an ESP32 node with an HC-SR04 sensor, SSD1306 OLED and a button. Each press sends one BLE reading to the Expo/React Native app. The app connects by scan/pick, persists settings and history, reconnects automatically and plays a phone-side alert. See §0.

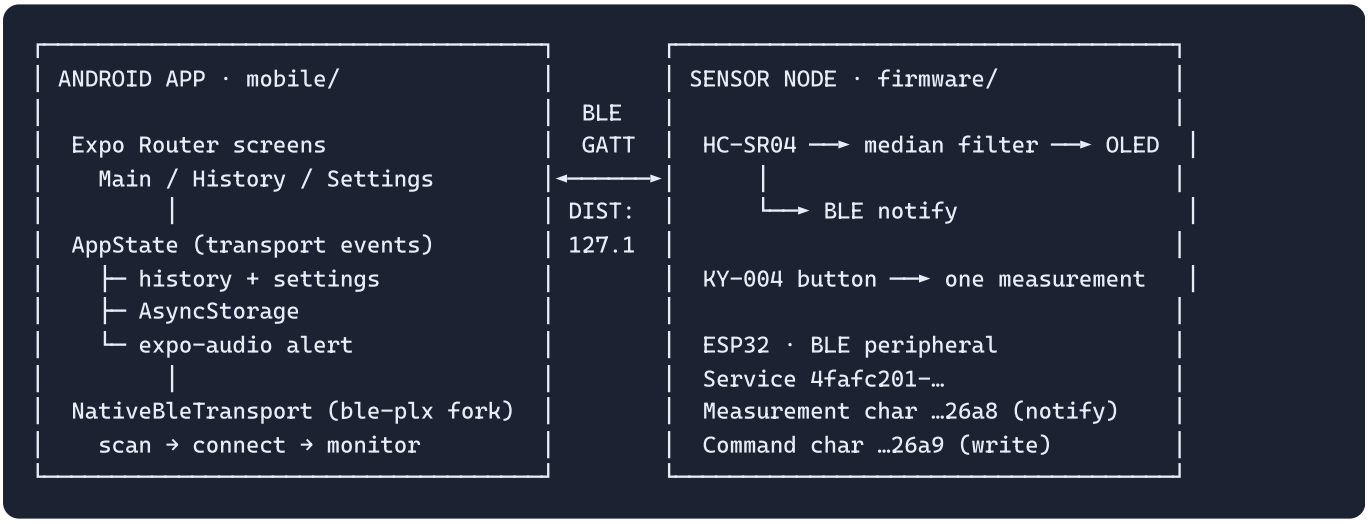
0 · Final application (v1.0)

This section supersedes the Milestone 1 mockup description. The final system is an ESP32 BLE sensor node plus an Android development build of the Expo app.

0.1 What changed from Milestone 1

Area	Milestone 1	Final v1.0
Controller	Arduino UNO	ESP32 Dev Module
Radio	HC-05 classic SPP	BLE GATT
Measurement	Continuous 5 Hz stream	One filtered reading per button press
Local UI	None	SSD1306 OLED with 10 s sleep
Alert	Buzzer on node	Phone-side Beep/Chime/Off
App data	Simulated feed, in-memory	Real BLE readings, AsyncStorage
Connection	None	Scan/pick, last-device restore, backoff reconnect

0.2 Architecture



0.3 App internals

- **Transport** — `src/services/bleTransport.ts` wraps `@sfourdrinier/react-native-ble-plx` 3.9.x, the maintained RN 0.86 / Expo SDK 57 fork. It scans client-side, connects with a timeout, discovers the service, reads the initial value, subscribes to notifications and reconnects with backoff.
- **Protocol** — `src/services/protocol.ts` decodes base64 BLE values and buffers partial lines. Unknown frames are ignored; `DIST:ERR` becomes a no-echo state and never a history row.
- **App state** — `src/state/AppState.tsx` owns the latest reading, history, settings, device list, connection state and the global device sheet. Every successful reading is capped at the 50-entry limit.
- **Persistence** — `src/storage/` validates settings, history and the last device, then stores them under versioned AsyncStorage keys.

- **Alerts** — `src/utils/alerts.ts` decides when a reading is below the threshold; `expo-audio` plays local WAV beep or chime assets.
- **Browser fallback** — `bleTransport.web.ts` keeps the UI usable but explains that BLE requires an Android development build.

0.4 Firmware internals

- Five samples per press; invalid echoes are discarded and the median is used. A configurable datum offset is added after filtering.
- The OLED shows idle, measuring, result, no-echo and BLE state. It sleeps after 10 s and wakes on the next press. A missing OLED does not block BLE.
- The node advertises as `DistanceMeter`, restarts advertising after a disconnect and answers `PING`.
- Every frame is mirrored to USB serial at 115200 for bench testing.
- The HC-SR04 ECHO line must pass through a 1 kΩ/2 kΩ divider. This is a hard hardware requirement: the ESP32 GPIOs are 3.3 V only.

0.5 Final BLE protocol

Direction	Frame	Meaning
Node → app	`DIST:<cm>`	One filtered reading
Node → app	`DIST:ERR`	No valid echo
Node → app	`INFO:MEASURING_TAPE_V2`	Identity / initial read
Node → app	`PONG`	Reply to PING
Node → app	`OK:CAL=<cm>`	Calibration accepted
Node → app	`ERR:<reason>`	Command rejected
App → node	`PING`	Liveness check
App → node	`CAL:<cm>`	Datum offset, −100…+100

Full GATT UUIDs and framing rules: `docs/PROTOCOL.md`.

0.6 Build and verification

- App: `npx expo run:android` or an EAS development build. BLE cannot run in Expo Go or a browser.
- Firmware: Arduino IDE, board **ESP32 Dev Module**, libraries Adafruit SSD1306 / GFX / BusIO.
- Gates: `tsc`, ESLint, Expo Doctor, Jest (26 tests), web export and Android prebuild all pass.
- Physical acceptance still requires the ESP32 and an Android phone: serial frames, nRF Connect GATT check, app scan/connect, button reading, persistence, reconnect, alert sound and accuracy against a steel tape.

> **HISTORICAL MATERIAL FOLLOWS**
> Sections 1–12 below are retained from the Milestone 1 documentation and describe the earlier UNO/HC-05 mockup. Use §0 and the Markdown docs for the current system.

1 · Historical project overview

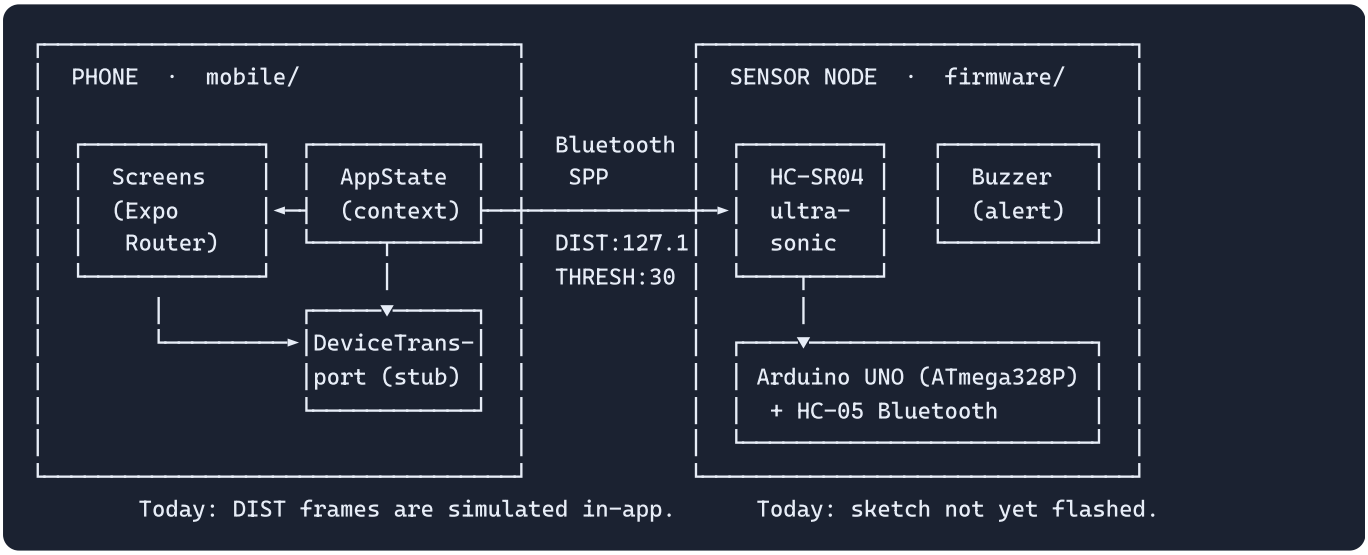
> **HISTORICAL**
> This section describes the original Milestone 1 UNO/HC-05 mockup. It is kept for project history; the final system is described in §0.

The Arduino Measuring Tape is a two-part system: a handheld sensor node that measures distance and a phone application that displays, logs and configures it.

1.1 What it does

- The **UNO node** measures distance with an HC-SR04 ultrasonic sensor and streams readings over a Bluetooth SPP link.
- The **phone app** shows the live reading on a gauge, keeps a measurement history, and lets the user choose units and alert thresholds.
- A **buzzer** on the node provides a local alert when a reading crosses the threshold.

1.2 System diagram



1.3 Repository layout

```
Arduino Measuring Tape/
├── README.md                    Project entry point
├── PROGRESS.md                  Milestone status report
├── documentation.pdf            This document
├── User Interface-1.pdf         Source design mockup
├── docs/
│   ├── documentation.html       Source of this PDF
│   └── UI-SPEC.md               Screen-by-screen design notes
├── firmware/
│   ├── README.md                Wiring + protocol reference
│   └── measuring_tape/
│       └── measuring_tape.ino   Arduino UNO sketch
└── mobile/                      Expo + TypeScript application
    ├── app.json                 Expo configuration
    ├── package.json             Dependencies & scripts
    ├── eslint.config.js         Lint configuration
    ├── tsconfig.json            TypeScript (with @/* → src/* alias)
    └── src/
        ├── app/                 Expo Router routes
        │   ├── _layout.tsx      Root: providers + stack
        │   └── (tabs)/
        │       ├── _layout.tsx  Bottom tab navigator
        │       ├── index.tsx    Main / Distance
        │       ├── history.tsx  Measurement History
        │       └── settings.tsx Settings
        ├── components/          Reusable presentational components
        ├── state/AppState.tsx   Single source of truth
        ├── services/device.ts   Domain types + DeviceTransport contract
        ├── data/                Seed measurements
        ├── utils/               Unit + time formatting
        └── theme/index.ts       Design tokens
```

2 · Getting started

2.1 Prerequisites

Tool	Version used	Notes
Node.js	24.18	Any LTS ≥ 20 works
npm	11.16	Ships with Node
Expo SDK	57	Resolved by `expo install`
Arduino IDE	2.x	Firmware only; AVR core 1.8.6+

2.2 Run the mobile app

```
cd mobile
npm install        # only needed once

npm run web        # browser preview  → http://localhost:8081
npm start          # dev server; press a (Android) or i (iOS)
npm run android    # Android emulator
npm run typecheck  # tsc --noEmit
npm run lint       # expo lint
```

> **PREVIEWING ON A PHONE**
> Install **Expo Go**, run `npm start`, and scan the QR code. The phone and the PC must be on the same network.

2.3 Flash the firmware

1. Open `firmware/measuring_tape/measuring_tape.ino` in the Arduino IDE.
2. Select **Tools** → **Board** → **Arduino UNO** and the correct port.
3. Upload. The sketch also mirrors frames to USB serial at 9600 baud for testing.

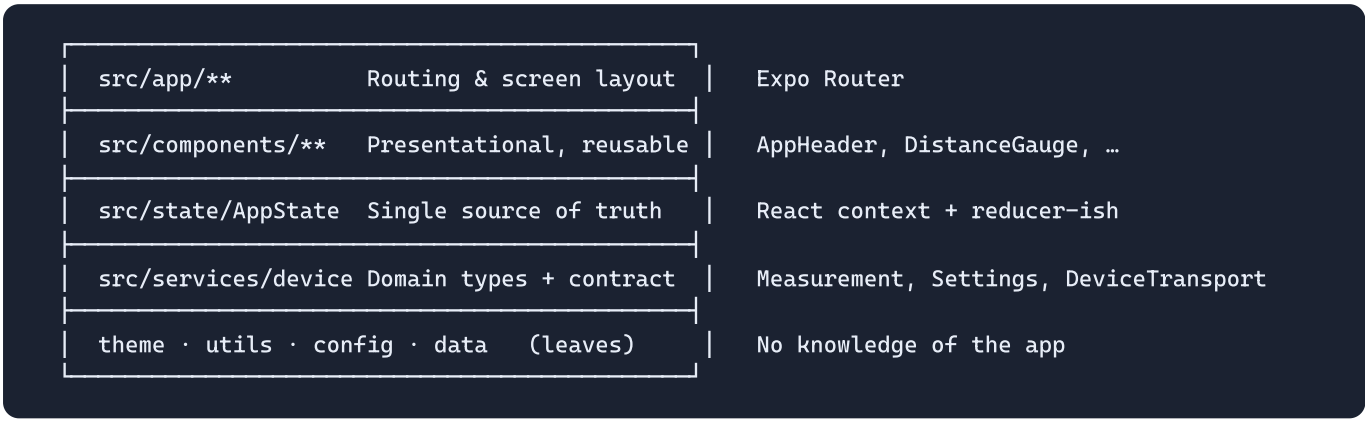
2.4 Configuration switches

Constant	File	Effect
`SIMULATE_LIVE_DATA`	`src/config.ts`	Set to `false` to stop the fake feed once real Bluetooth is wired in
`MAX_RANGE_CM`	`src/config.ts`	Full scale of the gauge (200 cm)
`HISTORY_LIMIT`	`src/config.ts`	Maximum stored measurements (50)
`LIVE_TICK_MS`	`src/config.ts`	Simulated sample interval (2000 ms)
`SENSOR_NOISE_CM`	`src/config.ts`	Simulated jitter (±0.9 cm)

3 · System architecture

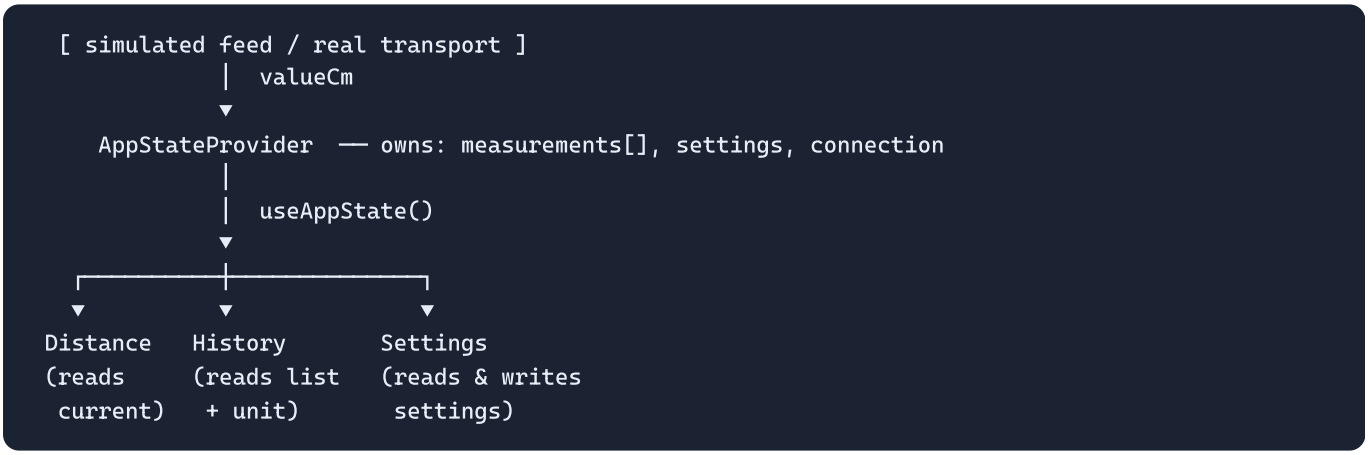
3.1 Layering

The mobile app is organised into four layers. Dependencies point strictly **downward**: screens compose components, components read state, state depends on domain types, and the leaves (theme, utils, config, data) depend on nothing.



The important property is that **no screen imports a hardware API**. Screens consume domain types and state; the transport is an implementation detail behind `DeviceTransport`.

3.2 Data flow



There is exactly **one** list of measurements. The live reading is simply `measurements[0]`, so the hero value and the top history row can never drift apart — there is no second "current value" field to keep in sync.

3.3 Key architectural decisions

Decision	Rationale / consequence
Single canonical unit (cm)	Every measurement is stored in centimetres; conversion happens only at render time. Changing units never mutates history.
One measurements array	Live value = `measurements[0]`. Removes a whole class of sync bugs.
Screens depend on `DeviceTransport` types, not BLE	Swapping the mock for real Bluetooth is additive; no screen edits required.
Design tokens, not literals	All colours, spacing and radii live in one file and can be re-themed globally.
Simulated feed behind a flag	The mockup is demonstrable today; the real feed replaces one effect.
Dependency-light interactions	Option sheets use the platform `Modal` instead of a picker library.

4 · Mobile application

4.1 Routing — Expo Router

Navigation is file-based. `src/app/` is the route root (Expo reports "*Using src/app as the root directory for Expo Router*" at startup). Non-route code lives outside `src/app/`, as Expo recommends.

File	Role
`src/app/_layout.tsx`	Root layout: `SafeAreaProvider` → `AppStateProvider` → `StatusBar` → headerless `Stack`
`src/app/(tabs)/_layout.tsx`	Bottom tab navigator with a themed bar and active-state pill
`src/app/(tabs)/index.tsx`	`/` — Main / Distance
`src/app/(tabs)/history.tsx`	`/history` — Measurement History
`src/app/(tabs)/settings.tsx`	`/settings` — Settings

4.2 State — `AppState`

The provider owns three pieces of state and exposes one hook:

```
useAppState() → {
  connection      : 'disconnected' | 'connecting' | 'connected'
  current         : Measurement | null      // = measurements[0]
  measurements    : Measurement[]           // newest first
  settings        : Settings
  clearHistory()  : void
  updateSettings(patch: Partial<Settings>) : void
}
```

A `useEffect` interval drives the simulated feed. Each tick copies the head entry with a jittered value and a fresh timestamp, and leaves the rest of the list untouched:

```
const drift = (Math.random() - 0.5) * SENSOR_NOISE_CM * 2;
const nextValue = clamp(round1(head.valueCm + drift), 0, MAX_RANGE_CM);
return [{ ...head, valueCm: nextValue, timestamp: Date.now() }, ...prev.slice(1)];
```

> **SEAM**
> Replacing the mock means deleting this effect and subscribing to `transport.subscribe` instead — the rest of the app is unaffected.

4.3 Domain — `services/device.ts`

Type	Shape
`Measurement`	`{ id: string; valueCm: number; timestamp: number }` — value always centimetres, time in epoch ms
`Settings`	`{ units: DistanceUnit; alertThresholdCm: number; alertSound: AlertSound }`
`DistanceUnit`	`'Centimeter' | 'Meter' | 'Inch' | 'Feet'`
`AlertSound`	`'Beep' | 'Chime' | 'Off'`
`ConnectionState`	`'disconnected' | 'connecting' | 'connected'`
`DeviceTransport`	`{ state; connect(); disconnect(); subscribe(listener) }`

`DeviceTransport` is the single contract the UI is written against. A real implementation wraps `react-native-ble-plx` (BLE) or a classic SPP bridge and requires no changes to screens.

4.4 Component catalogue

Component	Responsibility
`AppHeader`	Leading section icon + title, trailing connection dot and "Connected"/"Offline" label
`DistanceGauge`	Half-circle SVG dial; track plus gradient progress arc, sweep = `value / max`
`StatusBadge`	Pill: green "Within safe range" or amber "Below alert threshold"
`HistoryRow`	Gradient bar, value + unit, relative timestamp
`SectionCard` / `Divider`	Rounded surface container and hairline separator
`SelectRow`	Label + value with chevron; opens a dependency-free `Modal` option sheet
`ThresholdRow`	Numeric field, colour swatch and `cm` suffix
`ActionButton`	Pill button, `solid` or `outline` variant

4.5 Gauge geometry

The dial is a 180° arc drawn with `react-native-svg`. The arc path runs from the left edge, over the top, to the right edge; the progress arc uses `strokeDasharray` proportional to the arc length:

```
radius    = (size − strokeWidth) / 2
arcLength = π · radius
path      = M (cx − r) cy  A r r 0 0 1 (cx + r) cy
dash      = [ arcLength · (value / max), arcLength ]
```

4.6 Utilities

Module	Exports
`utils/units.ts`	`toDisplayDistance` (returns `{ value, text, suffix }`), `clamp`, `round1`, `UNIT_SUFFIX`
`utils/time.ts`	`formatClock` (`10:37:45`), `formatHistoryStamp` (`Today 21:38`)
`data/mockMeasurements.ts`	Eleven seed readings; the head (127.1 cm) is the one kept live

4.7 Tooling

- **TypeScript** — `strict: true`; path alias `@/* → src/*`.
- **ESLint** — `eslint-config-expo/flat`, run with `--max-warnings=0`.
- **expo-doctor** — validates the whole dependency graph (21 checks).

5 · Design system

Every value is centralised in `src/theme/index.ts`. Screens describe themselves with tokens rather than raw hex values, so the palette can be changed in one place.

5.1 Colour tokens

Token	Value	Used for
`background`	`#161B27`	App canvas
`surface`	`#212838`	Cards, tab bar
`surfaceAlt`	`#2A3242`	Gauge track, option sheet
`border`	`#2E3648`	Hairlines, outlines
`textPrimary`	`#EAF0FA`	Values, titles
`textSecondary`	`#8B94AA`	Field labels, units
`textMuted`	`#6A7390`	Timestamps, captions
`accent`	`#3AA0FF`	Primary button, gauge end stop
`accentBright`	`#22D3EE`	Active tab, header icons
`accentText`	`#7CC7FF`	Hero distance reading
`success`	`#34D399`	Connected dot, safe badge
`warning`	`#FBBF24`	Out-of-range badge

5.2 Gradients, spacing, radius, type

```
gradients.accent      = ['#5BE3FF', '#2F8BFF']   // gauge arc
gradients.historyBar  = ['#3FD8FF', '#1E3A5F']   // history row bar

spacing = { xs:4, sm:8, md:12, lg:16, xl:20, xxl:28 }
radius  = { sm:8, md:12, lg:18, xl:24, pill:999 }

typography.value      = 66px / 700   // hero reading
typography.headerTitle= 20px / 700
typography.caption    = 12px / 600, letter-spacing 2
typography.body       = 15px / 500
```

5.3 State colours

Condition	Badge	Colour
Reading ≥ alert threshold	Within safe range	green
Reading < alert threshold	Below alert threshold	amber
Device linked	Connected dot	green
Device offline	Offline dot	red

6 · Screens in detail

6.1 Main / Distance done

Route `/` · `src/app/(tabs)/index.tsx`

1. **Header** — Bluetooth icon + "Distance"; green dot + "Connected".
2. **Hero value** — the current reading at ~66 px in `accentText`, with the unit suffix in `textSecondary`.
3. **Caption** — `CURRENT DISTANCE`, letter-spaced, muted.
4. **Gauge** — half-circle dial; sweep = `value / MAX_RANGE_CM`.
5. **Status badge** — green or amber, driven by the alert threshold.
6. **Footer** — `Last updated: HH:MM:SS`, refreshed by the feed.

6.2 Measurement History done

Route `/history` · `src/app/(tabs)/history.tsx`

- Newest-first `FlatList` of readings.
- Each row: gradient bar, value + unit, right-aligned relative timestamp (`Today 21:38`, `Yesterday 10:27`, or a date).
- Hairline separators; empty state reads "No measurements yet. Connect the tape to start recording."

6.3 Settings done

Route `/settings` · `src/app/(tabs)/settings.tsx`

- **Units** — option sheet: Centimeter / Meter / Inch / Feet. Re-renders the hero value and every history row immediately.
- **Alert threshold** — numeric field (max 3 digits), colour swatch, `cm` suffix.
- **Alert sound** — option sheet: Beep / Chime / Off.
- **Clear History** — outlined pill, destructive confirmation dialog.
- **Save Settings** — solid accent pill.

6.4 Deviations from the mockup

Mockup	Implementation	Why
History/Settings headers both read "Distance"	"Measurement History" / "Settings"	The duplicate was a placeholder
Ambiguous filled dome gauge	Stroked half-circle arc with a value sweep	Reads clearly as a gauge and animates meaningfully
Palette given only as pixels	Centralised tokens	Consistency and theming
Static numbers	Simulated 2 s feed + seeded history	Demonstrates live behaviour pre-hardware

7 · Firmware — Arduino UNO (historical)

> **HISTORICAL**
> Superseded by the ESP32 firmware in §0 and `firmware/esp32_measuring_tape/`.

7.1 Bill of materials

Part	Purpose	Notes
Arduino UNO (ATmega328P)	Controller	5 V logic
HC-SR04	Ultrasonic distance	2–400 cm range
HC-05 / HC-06	Bluetooth SPP link	9600 baud default
Piezo buzzer	Local alert	Optional, D8

7.2 Pin map

Signal	UNO pin
HC-SR04 `TRIG`	D9
HC-SR04 `ECHO`	D10
Buzzer `+`	D8 (through 220 Ω)
HC-05 `TXD`	D2 — SoftwareSerial RX
HC-05 `RXD`	D3 — SoftwareSerial TX, via a voltage divider

> **HARDWARE CAUTION**
> The HC-05 `RXD` pin is a 3.3 V input. Feed it through a divider (1 kΩ series + 2 kΩ to GND); driving it directly from a 5 V pin degrades the module over time.

7.3 Sampling algorithm

```
loop():
  readCommands()                       // parse THRESH: / BEEP: / PING
  if (millis() - lastSampleAt < 200) return     // 5 Hz
  raw = readDistanceCm()               // pulseIn, 30 ms timeout
  if (raw < 0) { emit "DIST:ERR"; return }      // nothing in range
  smoothed += (raw - smoothed) * 0.35  // exponential moving average
  emit "DIST:" + smoothed (1 dp)
  applyAlert(smoothed)                 // buzzer below threshold

readDistanceCm():
  pulse TRIG high for 10 µs
  duration = pulseIn(ECHO, HIGH, 30000 µs)
  return duration · 0.0343 / 2         // cm, speed of sound
```

Why an EMA? Ultrasonic readings are noisy at the centimetre scale. `SMOOTHING = 0.35` weights each new sample at 35 %, damping spikes while staying responsive. It is the first value to tune on real hardware.

7.4 Sketch structure

Function	Responsibility
`setup()`	Pin modes, Serial + SoftwareSerial at 9600, boot banner
`loop()`	Superloop: commands, timed sampling, smoothing, emit, alert
`readDistanceCm()`	Trigger + timed echo; returns −1 on timeout
`readCommands()`	Reads one line from Bluetooth and dispatches
`handleCommand()`	Parses `THRESH:`, `BEEP:`, `PING`
`sendLine()`	Writes a frame to both Bluetooth and USB serial
`applyAlert()`	Drives the buzzer when below threshold

8 · Wire protocol — SPP (historical)

> **HISTORICAL**
> The final BLE protocol is in §0.5 and `docs/PROTOCOL.md`.

Newline-delimited ASCII, 9600 baud, 8N1. Human-readable so the link can be exercised with any serial monitor.

8.1 Device → phone

Frame	Meaning
`DIST:<cm>`	Smoothed distance, e.g. `DIST:127.1`
`DIST:ERR`	Echo timed out — nothing in range
`INFO:MEASURING_TAPE_V1`	Sent once on boot
`PONG`	Reply to `PING`
`OK:<key>=<value>`	Command acknowledged
`ERR:<reason>`	Command rejected

8.2 Phone → device

Frame	Meaning
`THRESH:<cm>`	Set the alert threshold, e.g. `THRESH:30`
`BEEP:<0|1>`	Enable / disable the buzzer
`PING`	Liveness check

8.3 Typical exchange

```
phone                  device
  |                      |
  |   (connect, SPP)     |
  |--------------------->|
  |                      |  INFO:MEASURING_TAPE_V1
  |<---------------------|
  |  THRESH:30           |
  |--------------------->|
  |                      |  OK:THRESH=30.0
  |<---------------------|
  |                      |  DIST:127.1      (every 200 ms)
  |<---------------------|
  |  BEEP:0              |
  |--------------------->|
  |                      |  OK:BEEP=0
  |<---------------------|
```

NOT YET IMPLEMENTED

The app does not speak this protocol today — the transport is a stub. The tables above are the contract the real transport must implement.

9 · Quality & verification

9.1 Quality gates

Command (from `mobile/`)	Result
`npx tsc --noEmit`	**0 errors**
`npx eslint . --max-warnings=0`	**0 problems**
`npx expo-doctor`	**21 / 21 checks**
Metro bundle — web	**bundled, 984 modules**
Metro bundle — Android	**bundled, 1605 modules**

9.2 What "verified" means here

- **Type & lint clean** — no type errors, no lint warnings.
- **Dependency graph healthy** — `expo-doctor` validates versions and peer deps.
- **Bundles build** — both web and native module graphs compile end to end.

> **NOT YET VERIFIED**
>
> No unit or component tests exist, and the firmware has not been flashed to physical hardware. Screenshots were not captured in the build environment (no browser was attached to the session); run `npm run web` to preview.

9.3 Notable environment fix

`react-dom` is pinned to **19.2.3** to match `react`. `expo-router` otherwise resolves `react-dom@19.3.0`, which requires `react@^19.3.0` and breaks npm's peer resolution.

10 · Progress report — Milestone 1

Field	Value
Milestone	1 — Foundation & mockup screens
Status	**Complete**
Date	1 October 2026
Deliverables	11 of 11 complete

10.1 Summary

The project now has a runnable foundation. The mobile app reproduces all three screens from `User Interface-1.pdf` — Distance, Measurement History and Settings — backed by a shared, typed app-state layer and a simulated sensor feed. The UNO firmware sketch is written and documented. Both halves are ready for the next milestone: replacing the mock transport with a real Bluetooth link.

10.2 Deliverables

#	Deliverable	Status	Where
1	Git repository + docs	done	repo root
2	Expo + TypeScript app with Expo Router	done	`mobile/`
3	Design tokens from the mockup	done	`src/theme/index.ts`
4	Main / Distance screen	done	`src/app/(tabs)/index.tsx`
5	Measurement History screen	done	`src/app/(tabs)/history.tsx`
6	Settings screen	done	`src/app/(tabs)/settings.tsx`
7	Shared UI components	done	`src/components/`
8	App state + simulated sensor feed	done	`src/state/AppState.tsx`
9	Device transport contract (stub)	done	`src/services/device.ts`
10	Arduino UNO firmware sketch	done	`firmware/measuring_tape/`
11	UI spec + firmware wiring docs	done	`docs/`, `firmware/README.md`

10.3 What works today

- **Navigation** — three tabs via Expo Router, themed bar with active-state highlight.
- **Live reading** — hero value, gauge sweep, badge and clock all track a simulated feed.
- **Units** — switching units immediately re-renders the hero value and every history row.
- **Alert rule** — dropping below the threshold flips the badge from green to amber.

- **History** — scrollable, newest-first, relative timestamps, clear action.
- **Settings** — option sheets, inline numeric threshold, save/clear actions.

10.4 Design decisions

1. **Expo Router + `src/app/`** — matches Expo guidance and keeps non-route code out of routes.
2. **Single canonical unit (cm)** — conversion at render only; history is immutable under unit changes.
3. **Design tokens over literals** — one place to adjust the palette.
4. **Dependency-light interactions** — platform `Modal` instead of a picker library.
5. **Transport contract first** — screens depend on `DeviceTransport` types, not BLE.

10.5 Known limitations

- Sensor feed is **simulated**; no Bluetooth implemented yet.
- Firmware is **untested on hardware**.
- `Alert`-based confirmations (`Save`, `Clear`) are no-ops on web.
- No persistence, tests, or reconnect/error states.
- `react-dom` pinned to 19.2.3 (see §9.3).

10.6 Next steps

1. **Bluetooth transport** — implement `DeviceTransport` (e.g. `react-native-ble-plx`) and parse `DIST:` frames.
2. **Persist history** — AsyncStorage or SQLite.
3. **Persist settings** — store preferences and push `THRESH:` on connect.
4. **Bench-test the firmware** — flash, verify accuracy, calibrate `SMOOTHING`.
5. **Device management UX** — pairing flow, reconnect handling, "searching" state.
6. **Polish** — empty/loading states, alert playback, unit-aware gauge labels, icons/splash.

11 · Roadmap

Milestone	Theme	Key work	Status
1	Foundation & mockup screens	Repo, Expo app, three screens, tokens, firmware sketch	done
2	Real connectivity	BLE/SPP transport, frame parsing, connection state, pairing UX	planned
3	Persistence & sync	Stored history, settings persistence, threshold push on connect	planned
4	Hardware bring-up & polish	Flash & calibrate, alerts, icons/splash, store builds	planned

11.1 Highest-risk items

- **Ultrasonic accuracy** — temperature/humidity sensitivity and surface angle; may need calibration or a median filter.
- **Bluetooth module choice** — classic SPP (HC-05) vs BLE changes the phone-side library; a decision to lock before Milestone 2.
- **iOS + BLE** — classic SPP is not available on iOS; a BLE module may be required for cross-platform support.

12 · Appendix

12.1 File index

Path	Purpose
`mobile/src/app/_layout.tsx`	Providers + headerless stack
`mobile/src/app/(tabs)/_layout.tsx`	Bottom tab navigator
`mobile/src/app/(tabs)/index.tsx`	Main / Distance screen
`mobile/src/app/(tabs)/history.tsx`	Measurement History screen
`mobile/src/app/(tabs)/settings.tsx`	Settings screen
`mobile/src/state/AppState.tsx`	App state provider + simulated feed
`mobile/src/services/device.ts`	Domain types + DeviceTransport contract
`mobile/src/components/*.tsx`	Eight reusable components
`mobile/src/theme/index.ts`	Design tokens
`mobile/src/utils/units.ts`	Unit conversion + numeric helpers
`mobile/src/utils/time.ts`	Clock + relative timestamp formatting
`mobile/src/config.ts`	Range, history limit, simulation switches
`mobile/src/data/mockMeasurements.ts`	Seed measurements
`firmware/measuring_tape/measuring_tape.ino`	Arduino UNO sketch

12.2 Glossary

Term	Definition
SPP	Serial Port Profile — classic Bluetooth serial link used by HC-05/HC-06
BLE	Bluetooth Low Energy — the alternative radio stack modern phones support
EMA	Exponential Moving Average — smoothing filter applied to sensor readings
Expo Router	File-based routing layer for React Native, built on React Navigation
DeviceTransport	The app-side contract abstracting the physical link
Canonical unit	Centimetres — the single unit in which all measurements are stored

12.3 Repository status

```
Branch   : main
Commit   : "Add project foundation: Expo mockup screens + UNO firmware"
Worktree : clean
Gates    : typecheck ✅   lint ✅   expo-doctor 21/21 ✅
```
